# Applied Changes for Interview

## 1. Combining Signals + RxJS — the modern pattern

### Context: Why this change matters for your interview

The original implementation of `CampaignService` acted as a **stateful store**—meaning the service itself held data (arrays of campaigns and currencies) in its own memory. This forced the service to manage the lifecycle of that data, rather than just fetching it. This is a common pattern in older Angular applications (often relying on `BehaviorSubject`, which is a type of Observable that remembers the last emitted value and instantly sends it to new subscribers so they have the current state), but in Angular 21, the officially recommended pattern is: **RxJS for async/HTTP, Signals for state.**

If you are asked *"How do you manage state?"* or *"When do you use RxJS vs Signals?"*, this change is your concrete example:

- **RxJS** is used exclusively to bridge the gap to the backend (because `HttpClient` returns Observables and represents asynchronous events over time).
- **Signals** are used exclusively inside the components to hold the synchronous state that drives the UI.

### The Problem with the Original Approach

In the original code:

1. The service holds the state ( `campaigns = signal([])` ).
2. When you navigate away from `CampaignListComponent` and come back, the service still holds the old data, but `loadCampaigns()` is called again. Because the service is a singleton ( `providedIn: 'root'` , meaning there is only one instance of the service shared across the entire application), the component immediately reads the old cached array from the service, rendering it on screen for a split second, before the new data arrives and replaces it. This causes a jarring flash.
3. Because the service is a globally shared singleton, its `loading` and `error` flags are also shared across the whole app. If two different components trigger an HTTP load simultaneously, their loading states could override each other, leading to unpredictable UI bugs (e.g., Component A's loader spins forever because Component B finished loading and set `service.loading = false` ).

# The Solution: Stateless Services, Stateful Components

## 1. The Service (Refactored)

We refactored `CampaignService` to be entirely stateless. It now simply acts as an API client, returning cold `Observable` s directly from Angular's `HttpClient` .

**Before (Stateful Service):**

```
@Injectable({ providedIn: 'root' })
export class CampaignService {
  private readonly http = inject(HttpClient);

  readonly campaigns = signal<Campaign[]>([]);
  readonly loading = signal(false);
  // ...

  loadCampaigns(): void {
    this.loading.set(true);
    this.http.get<Campaign[]>('/api/campaigns').subscribe({
      next: (data) => {
        this.campaigns.set(data);
        this.loading.set(false);
      }
    });
  }
}
```

**After (Stateless Service):**

```
@Injectable({ providedIn: 'root' })
export class CampaignService {
  private readonly http = inject(HttpClient);

  // Now simply returns the cold Observable, leaving the subscription to the consumer
  getAll(): Observable<Campaign[]> {
    return this.http.get<Campaign[]>('/api/campaigns');
  }
}
```

*Interview Talking Point:* "I refactored the services to be stateless API wrappers. This makes them easier to test because we don't have to mock internal state behavior or memory, just the HTTP

responses. It also prevents race conditions where two components accidentally overwrite the same global loading flag, and it allows Angular's garbage collector to properly clean up the memory when the component is destroyed."

## 2. The Components (Refactored)

Now, the components manage their own local state via Signals. They subscribe to the `CampaignService` Observables to populate those Signals.

**Before (Component as a dumb view):**

```
export class CampaignListComponent implements OnInit {
  readonly service = inject(CampaignService);
  // Relied entirely on the global service state
  readonly campaigns = computed(() => this.service.campaigns());

  ngOnInit(): void {
    this.service.loadCampaigns();
  }
}
```

**After (Component managing its own state):**

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

export class CampaignListComponent implements OnInit {
  private readonly campaignService = inject(CampaignService);
  private readonly destroyRef = inject(DestroyRef);

  // 1. Local state managed via Signals
  readonly campaigns = signal<Campaign[]>([]);
  readonly loading = signal(false);
  readonly error = signal<string | null>(null);

  ngOnInit(): void {
    this.loading.set(true);

    // 2. Consume the Observable
    this.campaignService.getAll()
      // 3. Prevent memory leaks! Automatically unsubscribe when component is destroyed
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next: (data) => {
          // 4. Update local Signals synchronously
          this.campaigns.set(data);
          this.loading.set(false);
        },
        error: (err) => {
          this.error.set(err.message);
          this.loading.set(false);
        }
      });
  }
}
```

## Key Takeaways for the Interview

- **Separation of Concerns:** The service is now responsible *only* for fetching data. The component is responsible for managing the state of its own view.
- **Memory Management:** We introduced `takeUntilDestroyed(this.destroyRef)`. This is a crucial Angular 16+ addition. If an HTTP request is slow and the user navigates away before it finishes, this operator automatically unsubscribes, preventing memory leaks and avoiding errors where the callback tries to update a component that no longer exists on the screen.

- **Predictable State:** The `loading` and `error` signals are now scoped to the component. If the `CampaignListComponent` fails to load, it won't accidentally show an error banner in the `CampaignDetailComponent` as well.

# 2. Observability: Structured Logging & Correlation IDs

## Context: Answering "Post-Release & Troubleshooting" Questions

If asked *"How do you monitor an application in production?"* or *"How do you troubleshoot a bug a user reported?"*, you should talk about **Observability**. Log files filled with `console.log('User created: John')` are impossible to search effectively in large systems.

**The modern approach:**

1. **JSON Structured Logging:** Logs should be output as JSON objects instead of plain text. This allows log aggregators (like Datadog, ELK stack, or Google Cloud Logging) to index the fields for easy querying (e.g., `level: error AND userId: 123` ).
2. **Correlation IDs:** Every incoming HTTP request should be assigned a unique ID (a UUID). This ID acts as the master thread tying a distributed transaction together. If a user clicks a button that hits an API, which queries a Database, which triggers an Email Service, looking at *just* the Email logs tells you nothing about *who* clicked the button. A Correlation ID attached to every single log explicitly connects the entire chain.

## The Implementation in NestJS

We implemented this using `nestjs-pino` (a very fast JSON logger) and `nestjs-cls` (Continuation-Local Storage). Node.js is single-threaded, meaning multiple user requests interweave asynchronously. `nestjs-cls` safely shares state (like our UUID) across those asynchronous function calls without needing to manually pass a `req` object everywhere. Pino automatically asks `nestjs-cls` for the current Context's Correlation ID every time `Logger.log()` happens, ensuring the identical UUID is seamlessly attached to every single log emitted during that specific user's HTTP request.

### 1. The Interceptor (The Entry Point)

We created a global `LoggingInterceptor` . Its job is to:

1. Generate a `CorrelationID` as soon as the request hits the server.
2. Log when the request *starts* (along with the URL, Method, etc.).
3. Measure how long the request takes to process.
4. Log when the request *completes* (recording the `Duration` and `Status Code`).

### 2. The Global Logger Configuration

We replaced the default NestJS console logger with `nestjs-pino`. We configured Pino to automatically attach the `CorrelationID` string to every log payload.

*Interview Talking Point:* "If a user reports an error saving a campaign, I ask them for their session ID or look up the timestamp, find the error log, grab the Correlation ID attached to it, and filter *all* logs across the entire backend by that ID. I can instantly see the exact trace of function calls and SQL queries leading up to the crash."

# 3. Database Refactoring: Normalization & Soft Deletes

## Context: Answering "Database Design & Evolving Changing Requirements"

Interviews often probe your understanding of database design, especially how you handle relationships and data safety.

### 1. Moving from Strings to Foreign Keys (Normalization)

**The Problem:** Originally, the `Campaign` table just stored `clientName` as a plain text string. If a client rebranded from "Acme Corp" to "Acme Global", you would have to run a mass update query on the `Campaign` table. It was prone to typos and couldn't store additional client metadata (like their industry).

**The Solution:** We **normalized** the database.

1. We created a separate `Client` entity table.
2. We changed the `Campaign` table to point to a `clientId` relationship.

*Interview Talking Point:* "I normalized the database by extracting unstructured text fields like `clientName` into dedicated entities with Foreign Key relations. This ensures data integrity and prepares the system for evolving requirements—for example, if the business later says 'We want to

track how much revenue comes from clients in the Retail industry vs Tech', we are now equipped to easily add an `industry` column to the new `Client` table."

### 2. Adding State and Soft Deletes

**The Problem:** The original campaign had no concept of its lifecycle (is it running? is it a draft?), and if the user clicked "Delete", the record was permanently destroyed from the database ( `DELETE FROM campaigns WHERE id = 1` ). This permanently breaks historical reporting.

**The Solution:**

1. **Status Enum:** We added a `status` field using an Enum ( `DRAFT` , `ACTIVE` , `PAUSED` , `COMPLETED` ).
2. **Soft Deletes:** We added a `deletedAt` timestamp column. Instead of hard-deleting the row, TypeORM now updates the `deletedAt` column with the current date. Active queries automatically filter out rows where `deletedAt IS NOT NULL` .

*Interview Talking Point:* "Financial data should almost never be hard-deleted. I implemented Soft Deletes using `deletedAt` columns. If an account manager deletes a campaign by mistake, we can instantly restore it by clearing the timestamp, and our analytics team still has an accurate historical record of all past campaigns."

# 4. 3rd Party API Integration & Testing

## Context: Answering "How do you work with 3rd Party Vendors & APIs?"

Interviews often ask how you handle external data sources, especially when those sources are out of your control (they might go down, have rate limits, or return unexpected formats).

## The Implementation in NestJS

We integrated the free `api.frankfurter.app` foreign exchange rate API to fetch real-time currency conversions.

### 1. The HTTP Wrapper Context

Instead of using a raw `fetch` call, we leveraged NestJS's `@nestjs/axios` `HttpModule` to create a dedicated `ExchangeRateService`.

### 2. Error Handling & Fallbacks (RxJS)

We piped the external HTTP call through RxJS operators. The `pipe()` acts as an assembly line for the asynchronous data. We used `map` to transform the successful data (drilling down into the Axios wrapper so the component only gets the exact rate object it needs), and crucially, we placed a `catchError` block at the end of the pipe. If the HTTP call or the `map` throws an error, `catchError` intercepts it instead of crashing the component stream.

*Interview Talking Point:* "When integrating with 3rd party APIs, I always assume the external service might fail. I used RxJS `catchError` to catch any network errors from the Frankfurter API and return an empty `of({})` fallback observable. `of()` creates a brand new, succeeding stream that emits an empty object. This ensures that if the external API goes down, our internal endpoints don't crash but instead gracefully degrade by handing the frontend an empty object."

### 3. Robust Unit Testing (Mocks)

We wrote comprehensive Vitest unit tests for both the new `Client` entity and the `ExchangeRateService`.

*Interview Talking Point:* "I never want my unit tests making real network calls. I injected a mock `HttpService` into my test suite using Vitest's `vi.spyOn`. This allowed me to simulate both a successful 200 OK response and a simulated 500 API failure, proving that my fallback logic works correctly without actually hitting the network during CI/CD builds."

# 5. Environment & Database Isolation

## Context: Avoiding Configuration Conflicts

Whenever you run applications inside containers mapping ports and volumes to the host (like `3306` for MySQL), you run the risk of namespace collisions with other local projects.

## The Implementation

To ensure this interview-specific project does not interfere with the original `campaign-budget` source repository, we completely isolated its MySQL context.

1. **Docker Compose:** We changed the initialization parameter `MYSQL_DATABASE` from `campaign_budget` to a uniquely named `campaign_budget_interview`.
2. **Dynamic Volumes:** We used `docker-compose down -v` to destroy the legacy unnamed/shared volume and allow a localized clean startup.
3. **App Fallbacks:** We reconfigured the `app.module.ts` TypeORM configurations to target the new secure schema.

*Interview Talking Point:* "When standing up new internal environments, I ensure I implement environment variable overloading and proper database isolation to prevent cross-contamination. I configured a custom MySQL schema initialization step specifically for this application so that it runs silently alongside any previous legacy databases without colliding."

# 6. Global Exception Filters & Health Checks

## Context: Answering "Post Release & Server Stability"

Deploying code is only half the battle. Interviewers want to know you understand how applications survive in modern cloud orchestration tools (like Kubernetes) and how they protect themselves against malicious users.

## The Implementation in NestJS

### 1. Terminus Health Pipes

Load balancers need to know if the application is "Ready" to receive traffic. We implemented `@nestjs/terminus` to create a dedicated `/api/health` HTTP probe.

When triggered, it returns an aggregate payload. The `info` block contains the status of all health indicators that *successfully passed* (like the database). The `error` block contains any health indicators that *failed*. The `details` block is the combined summary of *both* passing and failing checks.

*Interview Talking Point:* "To demonstrate production readiness, I added a Health Check endpoint. A load balancer uses this to constantly ping our API and ensure it's healthy. I specifically included a TypeORM database probe—so if the MySQL instance ever goes down, the API instantly marks itself as 'unhealthy' and the load balancer safely stops sending it traffic until it recovers."

### 2. Global Exception Filters (Error Masking)

When code throws a 500 Internal Server error, inexperienced developers often accidentally serialize the stack trace directly to the HTTP response, which is a massive security hole. We implemented a `GlobalExceptionFilter`.

*Interview Talking Point:* "I'm very strict about error handling. I built a `GlobalExceptionFilter` that catches every unhandled exception in the application. It masks the internal stack trace so hackers can't see our directory structures, but it logs the full error safely inside our server logs alongside the Correlation ID. If a user sees a generic '500 Server Error' message, they simply give us the Correlation ID attached to the response payload, and we can look up the exact line of code that failed in our secure internal logs."

# 7. Atomic Database Transactions

## Context: Answering "How do you ensure data consistency?"

A major requirement when designing a database is ensuring that split operations either completely succeed or completely fail together.

**The Problem:** Originally, `CampaignService.create()` worked by saving the `Campaign` row to the database, and then firing off a separate command to `DistributionService.distribute()` to create the budget allocations in the `CampaignChannel` table.
If the application crashed exactly halfway between those two operations, the system would be left with an "orphaned" campaign possessing a $10,000 budget but zero mathematical channel distribution.

## The Implementation in NestJS

We utilized TypeORM's `DataSource.transaction()` to bind the operations together atomically.

1. We injected the global `DataSource` into the `CampaignService`.
2. For the `create()`, `update()`, and `redistribute()` methods, we executed the internal logic inside an `await this.dataSource.transaction(async (manager) => { ... })` callback.

3. We updated `DistributionService.distribute()` to optionally accept that same `EntityManager`.

By passing the master transaction's `EntityManager` down into the child service, we ensure all nested `save()` and `delete()` SQL calls operate on the exact same active transaction wrapper.

*Interview Talking Point:* "Data integrity is critical when dealing with financial campaigns. I refactored the campaign lifecycle hooks to execute entirely within an atomic TypeORM `DataSource` transaction. This guarantees consistency: either the campaign *and* its subsequent budget distributions are all saved securely to MySQL as one un-interruptable block, or if any part of the math fails, the entire transaction rolls back instantly, completely eliminating the risk of corrupt, orphaned campaign records."